ĐỒ ÁN TỐT NGHIỆP

NỘI DUNG 1

So sánh LangChain RAG thuần và LangGraph Self-RAG

Faithfulness, answer relevancy và hallucination rate trên bộ bài báo khoa học

Tháng 10 năm 2026

1. MỤC TIÊU VÀ PHẠM VI

Nội dung 1 so sánh hai hệ thống hỏi đáp trên cùng bộ 95 bài báo khoa học: **LangChain RAG thuần** và **LangGraph Self-RAG**. Ba chỉ số so sánh là **faithfulness**, **answer relevancy** và **hallucination rate**. Toàn bộ hệ thống (mô hình sinh, judge, embedding model, cơ sở dữ liệu vector) chạy cục bộ, không dùng API bên ngoài.

Để so sánh có ý nghĩa, đánh giá được thực hiện trên ba loại câu hỏi (một đoạn, tổng hợp nhiều bài, out-of-corpus) và hai điều kiện truy xuất (top-k = 4 và top-k = 8 với nhiều đoạn không liên quan); faithfulness của hai hệ được chấm trên cùng một tập tài liệu.

2. MÔI TRƯỜNG VÀ REPRODUCIBILITY

Bảng 1. Môi trường thực nghiệm

Thành phần	Version / cấu hình
Python	3.14.6
langchain / langchain-core	1.4.3 / 1.6.6
langgraph	1.2.12
langchain-ollama / langchain-chroma	1.1.0 / 1.1.0
chromadb	1.5.9
Ollama server	0.35.0
LLM sinh câu trả lời	qwen2.5:7b (Q4_K_M, digest 845dbda0ea48)
Judge	llama3.1:8b (Q4_K_M, digest 46e0c10c039e)
Embedding model	nomic-embed-text (F16, digest 0a109f422b47)
Phần cứng	CPU i3-12100F, RAM 16 GB, GPU GTX 1660 Ti 6 GB

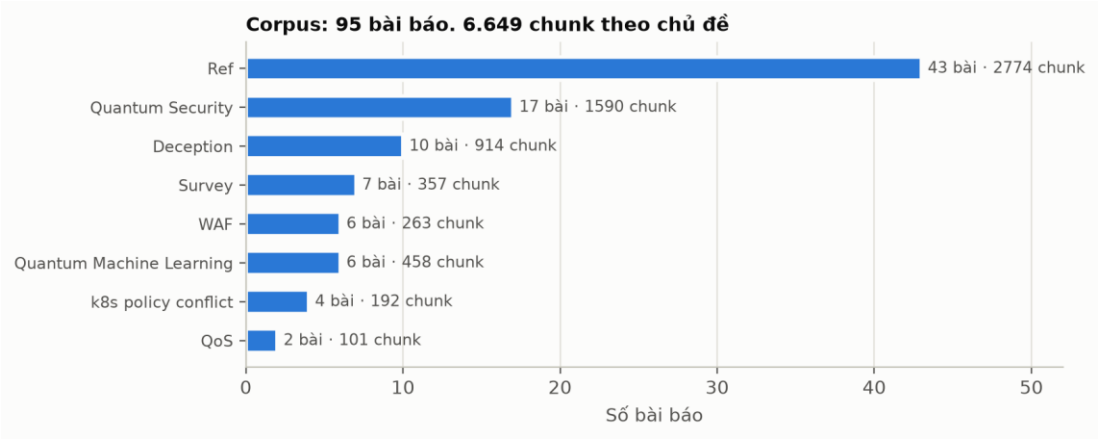
Trong quá trình chạy, Ollama tự cập nhật từ 0.35.0 lên 0.35.1; phần cuối của điều kiện top-k = 8 chạy trên 0.35.1, digest của ba mô hình không đổi. Version của từng lần chạy được ghi trong `results/<run>/run_info.json`.

Judge được chọn khác họ với mô hình sinh để tránh thiên lệch "tự chấm bài mình". Cả hai mô hình chạy với temperature = 0 và seed cố định. Do GPU chỉ có 6 GB, các mô hình đều là bản 7–8B lượng tử hoá Q4.

Mã nguồn, bộ câu hỏi, kết quả và dữ liệu đều có trong repository: `data/chunks.jsonl` (văn bản các chunk), `data/chroma.zip` (cơ sở dữ liệu vector đúng như lúc chạy, tự giải nén ở lần chạy đầu), `data/MANIFEST.json` (số lượng, mã sha256, tham số chunking, version). `requirements.txt` ghi đúng version thư viện và mỗi lần đánh giá lưu lại version môi trường trong `results/<run>/run_info.json`. Sau khi cài Ollama và tải ba mô hình, chạy lại toàn bộ đánh giá bằng `python evaluate.py`.

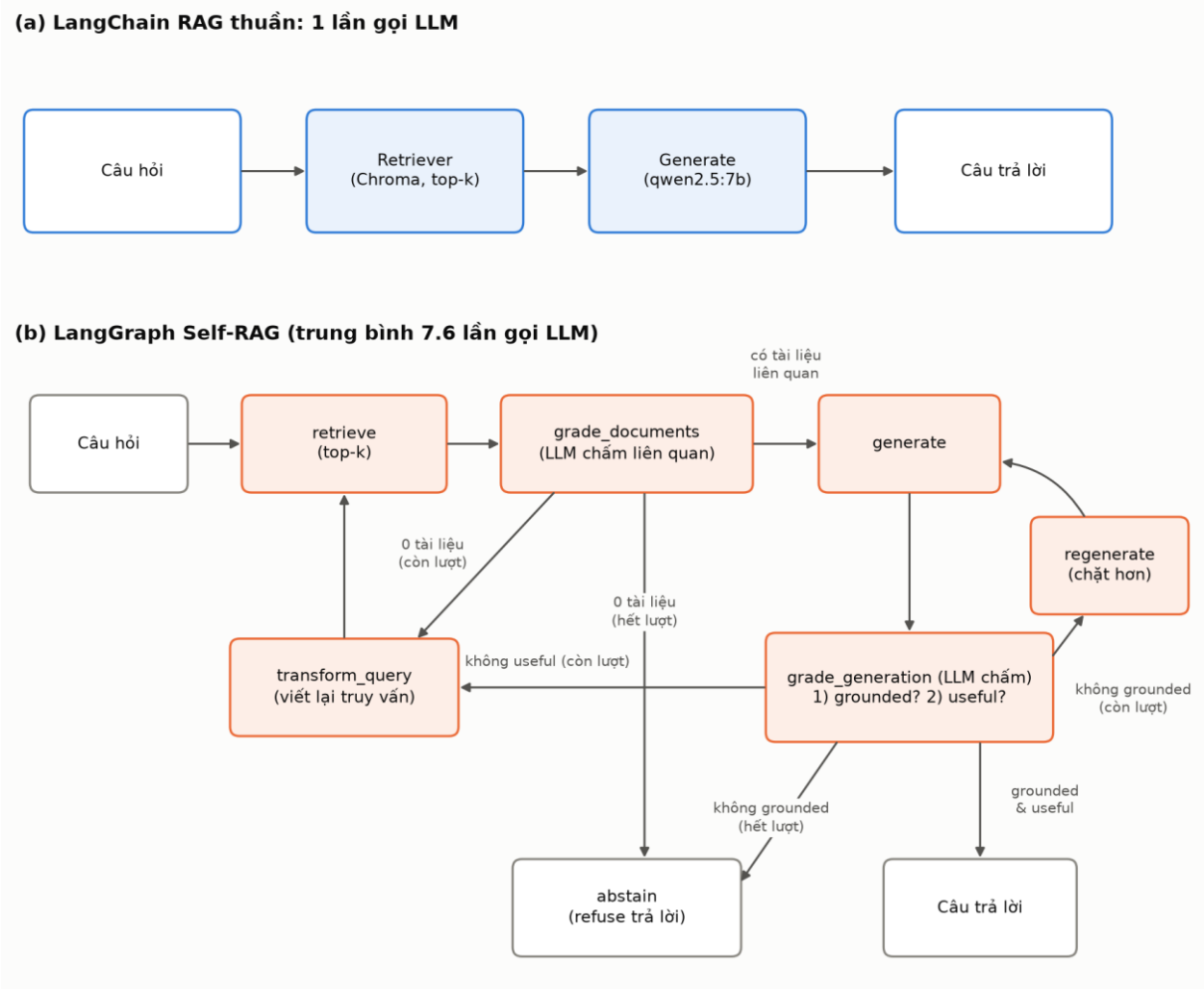
3. DỮ LIỆU

Từ 96 tệp PDF trong 7 chủ đề, văn bản được trích bằng PyMuPDF, loại phần tài liệu tham khảo cuối bài và 1 tệp trùng lặp, chia thành đoạn 1000 ký tự (chồng lấn 150) rồi nhúng bằng nomic-embed-text vào Chroma (khoảng cách cosine), thu được 95 bài báo và 6.649 chunk.



Hình 1. Phân bố bài báo và chunk theo chủ đề

4. HAI HỆ THỐNG



Hình 2. Kiến trúc (a) LangChain RAG thuần và (b) LangGraph Self-RAG

Hai hệ dùng chung dữ liệu, bộ truy xuất, top-k, mô hình sinh và prompt sinh câu trả lời. Prompt sinh câu trả lời yêu cầu chỉ trả lời dựa trên tài liệu và trả lời đúng một câu cố định khi tài liệu không có thông tin. Khác biệt duy nhất giữa hai hệ là vòng tự đánh giá của Self-RAG.

- **Hệ A - LangChain RAG thuần:** truy xuất top-k chunk rồi sinh câu trả lời; 1 lần gọi LLM mỗi câu hỏi.
- **Hệ B - LangGraph Self-RAG:** dựng bằng LangGraph theo luồng: truy xuất → chấm độ liên quan từng tài liệu → sinh câu trả lời → kiểm tra câu trả lời có căn cứ (grounded) → kiểm tra câu trả lời có hữu ích (useful). Không có tài liệu liên quan thì viết lại truy vấn, hết lượt thì refuse; không có căn cứ thì sinh lại với prompt chặt hơn, hết lượt thì refuse; có căn cứ nhưng không hữu ích thì viết lại truy vấn, hết lượt thì giữ câu trả lời (tối đa 2 lần viết lại truy vấn và 2 lần sinh lại cho mỗi câu hỏi).

Hệ B là **approximate / prompt-based Self-RAG**. Self-RAG gốc (Asai và cộng sự, 2023) huấn luyện mô hình để tự sinh các reflection token [Retrieve], [IsRel], [IsSup],

[IsUse]. Ở đây dùng mô hình qwen2.5:7b có sẵn, mỗi bước tự đánh giá là một lần gọi LLM với prompt chấm điểm có/không, và hệ luôn truy xuất (không có bước [Retrieve]):

Bảng 2. Đối chiếu Self-RAG gốc và hệ B

Self-RAG gốc (Asai và cộng sự, 2023)	Hệ B (LangGraph)
[Retrieve]: mô hình tự quyết định có cần truy xuất	Luôn truy xuất
[IsRel]: token đánh giá độ liên quan của passage	grade_documents: LLM chấm từng tài liệu liên quan / không
[IsSup]: token đánh giá câu trả lời có căn cứ	grade_generation: LLM chấm grounded (prompt có few-shot)
[IsUse]: token đánh giá độ hữu ích (thang 1–5)	grade_generation: LLM chấm useful (có / không)
Mô hình được fine-tune để sinh token	Mô hình có sẵn, không huấn luyện thêm

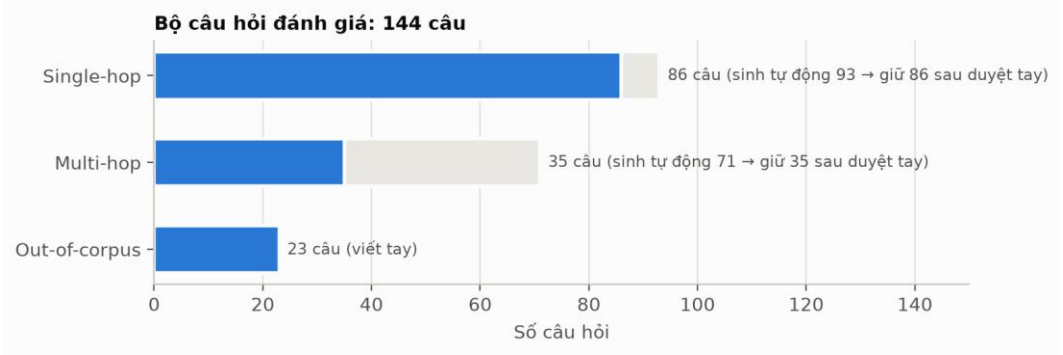
Mô hình 7B dễ chấm bước kiểm tra căn cứ (ISSUP) theo kiểu "câu trả lời có đủ ý không" thay vì "có bịa không", dẫn tới refuse hoặc sinh lại không cần thiết. Vì vậy prompt ISSUP định nghĩa rõ: câu trả lời có căn cứ khi mọi ý suy ra trực tiếp được từ tài liệu, **không cần chứa đủ mọi chi tiết**; chỉ chấm "không" khi có thông tin không có trong tài liệu hoặc mâu thuẫn với tài liệu. Prompt kèm 4 ví dụ few-shot (2 có căn cứ, 2 không), lấy ngoài lĩnh vực của corpus để không trùng với bộ câu hỏi đánh giá.

5. BỘ CÂU HỎI ĐÁNH GIÁ

Bảng 3. Ba loại câu hỏi

Loại	Số câu	Cách tạo	Mục đích
Single-hop	86	LLM sinh 1 câu hỏi + đáp án từ 1 đoạn ngẫu nhiên của mỗi bài (93 câu), duyệt tay	Câu hỏi sự kiện cơ bản
Multi-hop	35	Ghép 1 đoạn của bài A với đoạn gần nghĩa nhất của bài B; LLM đặt câu hỏi cần cả hai bài; loại câu mà 1 đoạn đã trả lời đủ (71 câu), duyệt tay	Câu hỏi tổng hợp, cần truy xuất nhiều nguồn
Out-of-corpus	23	Viết tay, chủ đề gần với corpus; kiểm tra từ khoá chứa đáp án không xuất hiện trong chunks	Hệ thống phải refuse trả lời (abstain)

Khi duyệt tay, mỗi câu được đối chiếu với passage gốc: sửa câu hỏi cho rõ và tự đứng được, sửa đáp án cho đúng với passage; loại câu hỏi về công thức bị lỗi ký tự, câu chỉ hỏi tên tác giả, và với multi-hop là các câu ghép hai ý không liên quan hoặc thực chất chỉ cần một bài. Lý do loại và nội dung sửa của từng câu được lưu trong `testset/review_edits.py` và `testset/review_multihop.py`. Ngoài ba loại câu hỏi, điều kiện **retrieval nhiễu** được tạo bằng cách chạy lại câu single-hop và multi-hop với top-k = 8.



Hình 3. Bộ câu hỏi đánh giá

6. PHƯƠNG PHÁP ĐÁNH GIÁ

Bảng 4. Ba chỉ số so sánh (định nghĩa theo RAGAS)

Chỉ số	Cách tính
Faithfulness ↑	Judge tách câu trả lời thành các claim, kiểm tra từng claim có suy ra được từ tài liệu. Điểm = số claim được hỗ trợ / tổng số claim. Câu refusal không có claim nên không tính.
Answer relevancy ↑	Judge tạo 3 reverse-generated question từ câu trả lời; điểm = độ tương đồng cosine trung bình với câu hỏi gốc. Câu né tránh / refuse được 0 điểm.
Hallucination rate ↓	Tỉ lệ câu hỏi có câu trả lời chứa ít nhất một claim không được tài liệu hỗ trợ.

- **Chấm trên cùng tài liệu:** Self-RAG lọc bớt tài liệu trước khi sinh câu trả lời. Nếu chấm faithfulness trên tài liệu đã lọc thì Self-RAG bị chấm trên ít bằng chứng hơn RAG thuần. Vì vậy faithfulness và hallucination của cả hai hệ được chấm trên full top-k ban đầu của câu hỏi gốc.
- **Câu out-of-corpus:** hành vi đúng là refuse, nên chỉ số cần xem là hallucination rate; answer relevancy của câu refusal bằng 0 theo định nghĩa nên không dùng để đánh giá loại câu này.
- **Thống kê:** mỗi chỉ số có CI 95% (bootstrap 2000 lần). Paired difference giữa hai hệ được kiểm định trên cùng câu hỏi (bootstrap); hallucination dùng thêm kiểm định McNemar.
- **Secondary metric:** refusal rate, độ đúng so với ground truth, retrieval hit, thời gian và số lần gọi LLM, chỉ dùng để giải thích kết quả.

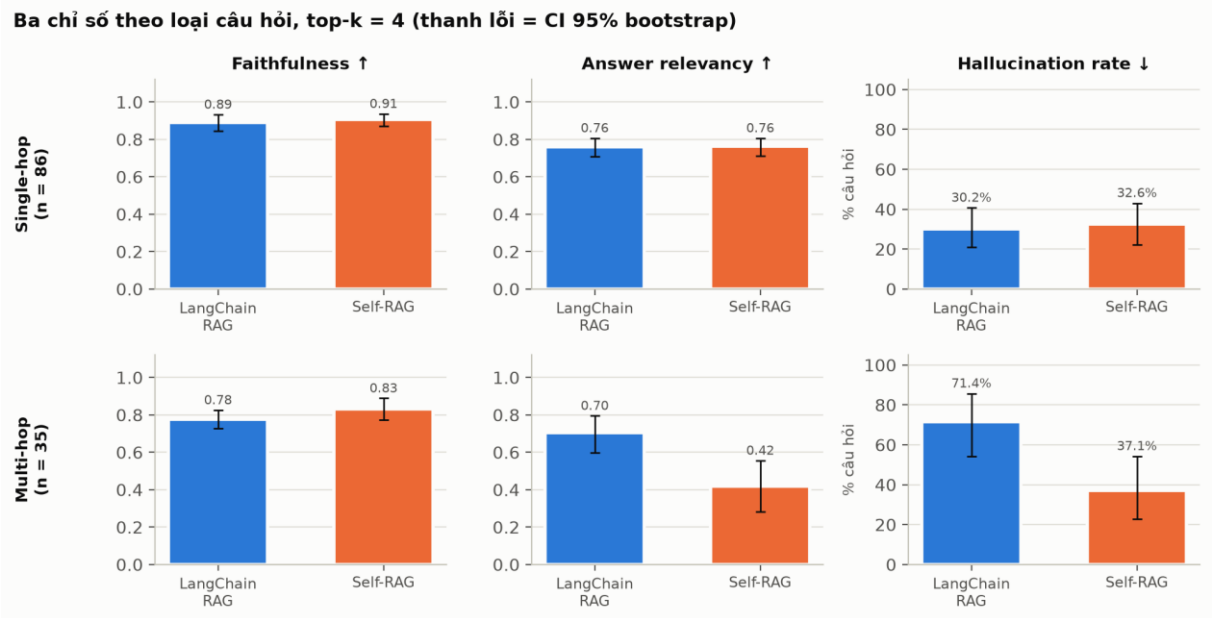
7. KẾT QUẢ

7.1. Ba chỉ số với top-k = 4

Bảng 5. Ba chỉ số, top-k = 4: giá trị trung bình [CI 95%]

Loại câu hỏi	Hệ thống	Faithfulness ↑	Answer relevancy ↑	Hallucination rate ↓
Single-hop (n = 86)	LangChain RAG	0,890 [0,845; 0,933] (n = 79)	0,759 [0,707; 0,807]	30,2% [20,9%; 40,7%]
	LangGraph Self-RAG	0,905 [0,871; 0,937] (n = 80)	0,763 [0,712; 0,807]	32,6% [22,1%; 43,0%]
Multi-hop (n = 35)	LangChain RAG	0,776 [0,727; 0,826] (n = 30)	0,703 [0,598; 0,795]	71,4% [54,3%; 85,7%]
	LangGraph Self-RAG	0,831 [0,775; 0,889] (n = 18)	0,419 [0,282; 0,556]	37,1% [22,9%; 54,3%]
Out-of-corpus (n = 23)	LangChain RAG	– (không có claim)	0,000 [0,000; 0,000]	0,0% [0,0%; 0,0%]
	LangGraph Self-RAG	– (không có claim)	0,000 [0,000; 0,000]	0,0% [0,0%; 0,0%]

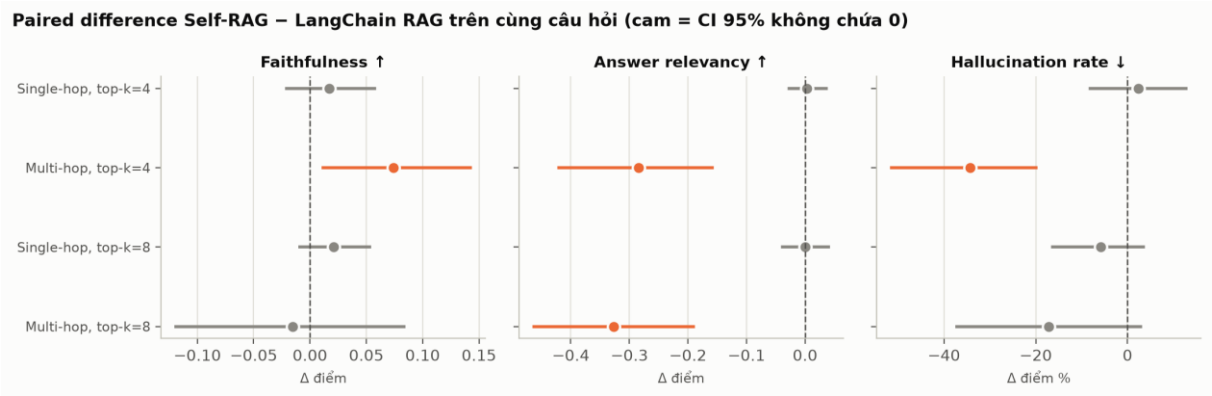
n ở cột đầu là số câu hỏi. Faithfulness chỉ tính trên câu trả lời có claim (bỏ câu refusal), n ghi trong ô; answer relevancy và hallucination rate tính trên đủ n câu (câu refusal có answer relevancy = 0, hallucination = 0), nên faithfulness và hallucination rate không suy ra được từ nhau.



Hình 4. Ba chỉ số theo loại câu hỏi, top-k = 4

Bảng 6. Paired difference Self-RAG − LangChain RAG, top-k = 4

Loại câu hỏi	Δ Faithfulness	Δ Answer relevancy	Δ Hallucination rate
Single-hop	+0,017 (p ≈ 0,39)	+0,003 (p ≈ 0,87)	+2,3 điểm % (p ≈ 0,75)
Multi-hop	+0,074 (p ≈ 0,01)	-0,284 (p < 0,001)	-34,3 điểm % (p < 0,001)



Hình 5. Paired difference trên cùng câu hỏi (màu cam: CI 95% không chứa 0)

Single-hop: không đủ bằng chứng để phân biệt hai hệ trên cả ba chỉ số (faithfulness 0,890 so với 0,905, answer relevancy 0,759 so với 0,763, hallucination rate 30,2% so với 32,6%); không paired difference nào có statistical significance ($p \approx 0,39$, $p \approx 0,87$, $p \approx 0,75$). Theo từng câu, Self-RAG sửa được 9 câu RAG thuần bị hallucination nhưng phát sinh 11 câu mới. Với loại câu hỏi dễ này, RAG thuần đã truy xuất đúng bài báo ở 97% số câu, nên vòng tự đánh giá có ít việc để cải thiện.

Multi-hop: Self-RAG có hallucination rate thấp hơn rõ rệt (37,1% so với 71,4%, $p < 0,001$; McNemar: 12 câu chỉ RAG thuần bị hallucination, 0 câu chỉ Self-RAG, $p < 0,001$) và faithfulness cao hơn (0,831 so với 0,776, $p \approx 0,01$, tính trên 18 câu cả hai hệ cùng trả lời). Ngược lại answer relevancy thấp hơn rõ rệt (0,419 so với 0,703, $p < 0,001$) vì Self-RAG refuse 49% số câu (RAG thuần 14%).

7.2. Câu hỏi out-of-corpus và refuse đúng

Bảng 7. Refuse đúng và refuse oan, top-k = 4

Hệ thống	Câu out-of-corpus được refuse (đúng)	Câu có đáp án bị refuse (oan)
LangChain RAG	23/23	12/121 (9,9%)
LangGraph Self-RAG	23/23	23/121 (19,0%)

Cả hai hệ đều refuse đúng 23/23 câu out-of-corpus, hallucination rate bằng 0%. Prompt sinh câu trả lời dùng chung đã yêu cầu trả lời "không tìm thấy" khi tài liệu không có thông tin và qwen2.5:7b tuân thủ tốt, kể cả với những câu mà corpus có nhắc tới entity liên quan (ví dụ hỏi số máy bị WannaCry lây nhiễm, trong khi corpus chỉ nhắc tên WannaCry). Như vậy với prompt có chỉ dẫn refuse rõ ràng, cơ chế abstain của Self-RAG không tạo thêm khác biệt ở loại câu hỏi này. Khác biệt nằm ở vế còn lại của "refuse đúng": Self-RAG refuse 23 câu có đáp án, gần gấp đôi RAG thuần (12 câu), chủ yếu ở câu multi-hop.

7.3. Retrieval nhiễu (top-k = 8)

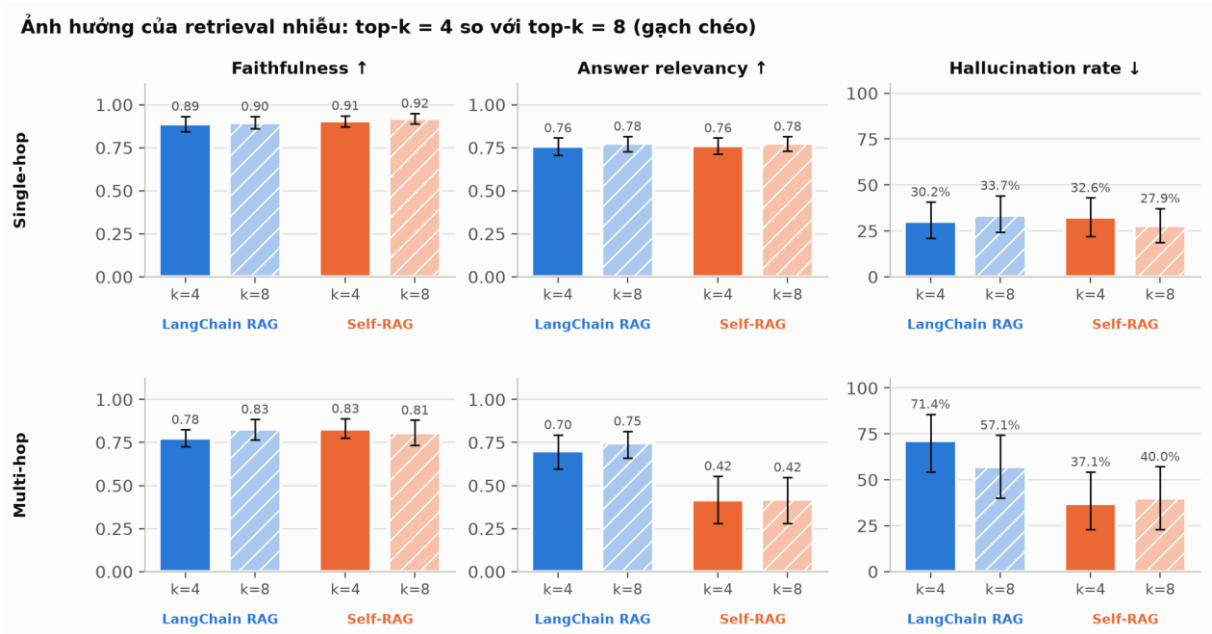
Bảng 8. Ba chỉ số, top-k = 8: giá trị trung bình [CI 95%]

Loại câu hỏi	Hệ thống	Faithfulness ↑	Answer relevancy ↑	Hallucination rate ↓
Single-hop (n = 86)	LangChain RAG	0,898 [0,860; 0,932] (n = 81)	0,776 [0,729; 0,816]	33,7% [24,4%; 44,2%]
	LangGraph Self-RAG	0,920 [0,889; 0,950] (n = 81)	0,776 [0,731; 0,816]	27,9% [18,6%; 37,2%]
Multi-hop (n = 35)	LangChain RAG	0,830 [0,767; 0,885] (n = 32)	0,747 [0,661; 0,816]	57,1% [40,0%; 74,3%]
	LangGraph Self-RAG	0,806 [0,733; 0,880] (n = 18)	0,420 [0,280; 0,548]	40,0% [22,9%; 57,1%]

n ở cột đầu là số câu hỏi. Faithfulness chỉ tính trên câu trả lời có claim (bỏ câu refusal), n ghi trong ô; answer relevancy và hallucination rate tính trên đủ n câu (câu refusal có answer relevancy = 0, hallucination = 0), nên faithfulness và hallucination rate không suy ra được từ nhau.

Bảng 9. Paired difference Self-RAG − LangChain RAG, top-k = 8

Loại câu hỏi	Δ Faithfulness	Δ Answer relevancy	Δ Hallucination rate
Single-hop	+0,021 (p ≈ 0,16)	+0,001 (p ≈ 0,97)	-5,8 điểm % (p ≈ 0,32)
Multi-hop	-0,015 (p ≈ 0,77)	-0,327 (p < 0,001)	-17,1 điểm % (p ≈ 0,12)



Hình 6. Ba chỉ số khi top-k = 4 và top-k = 8

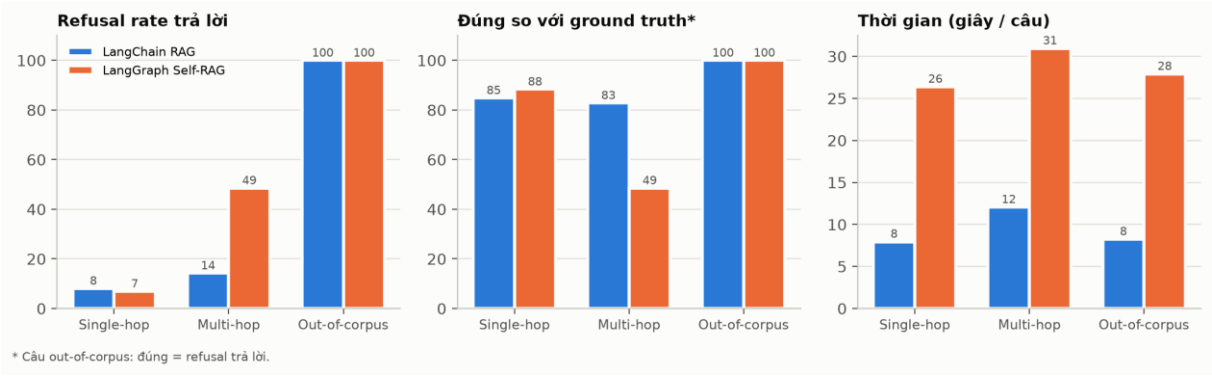
Single-hop: khi thêm đoạn nhiễu, hallucination rate của RAG thuần tăng nhẹ (30,2% → 33,7%) còn của Self-RAG giảm (32,6% → 27,9%); Self-RAG có faithfulness cao hơn (0,920 so với 0,898). Xu hướng này đúng với vai trò lọc tài liệu của Self-RAG nhưng chưa có statistical significance (p ≈ 0,32, p ≈ 0,16); answer relevancy trung bình của hai hệ trùng nhau (0,776), không đủ bằng chứng để phân biệt (p ≈ 0,97). Thay đổi

giữa top-k = 4 và top-k = 8 của từng hệ là hai lần chạy khác nhau, chỉ mô tả, không kiểm định.

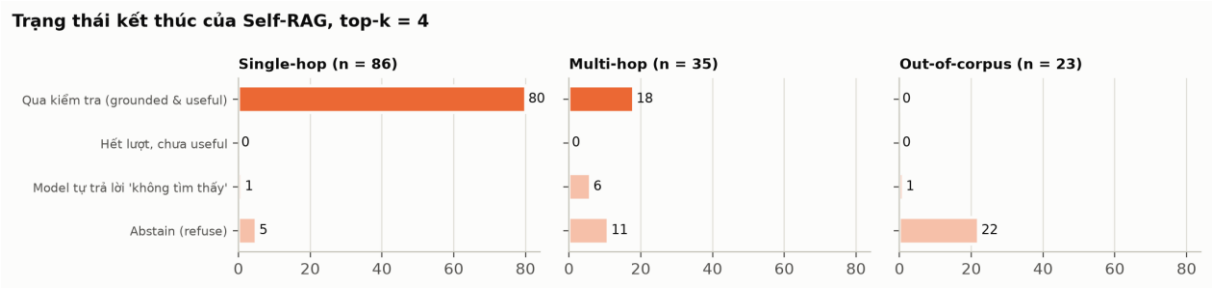
Multi-hop: top-8 giúp RAG thuần có đủ cả hai bài nguồn nhiều hơn (60% → 77% số câu), nên hallucination rate của RAG thuần giảm từ 71,4% xuống 57,1% và độ đúng tăng lên 91%. Self-RAG không được lợi tương tự: bước chấm độ liên quan vẫn loại mất một trong hai bài (tài liệu đưa vào sinh câu trả lời chỉ đủ hai nguồn ở 26% số câu) và hệ vẫn refuse 49% số câu. Chênh lệch hallucination rate còn 17,1 điểm % và không còn statistical significance ($p \approx 0{,}12$); faithfulness của hai hệ không đủ bằng chứng để phân biệt ($p \approx 0{,}77$); answer relevancy của Self-RAG vẫn thấp hơn rõ rệt (0,420 so với 0,747, $p < 0{,}001$).

Nguyên nhân là bước grade_documents chấm **từng đoạn riêng lẻ** với câu hỏi: với câu hỏi cần ghép thông tin của hai nguồn, đoạn chỉ chứa một nửa thông tin dễ bị chấm là không liên quan. Đây là giới hạn cụ thể của cách lọc tài liệu hiện tại đối với câu hỏi tổng hợp.

7.4. Phân tích bổ sung



Hình 7. Refusal rate, độ đúng so với ground truth và thời gian xử lý, top-k = 4



Hình 8. Trạng thái kết thúc của Self-RAG theo loại câu hỏi, top-k = 4

Vì sao Self-RAG refuse nhiều ở câu multi-hop: câu multi-hop cần thông tin của hai bài, nhưng top-4 thường chỉ chứa một bài (RAG thuần có đủ cả hai bài nguồn trong tài liệu ở 60% số câu). Bước chấm độ liên quan của Self-RAG giữ lại trung bình 1,4 tài liệu, nên tài liệu đưa vào sinh câu trả lời chỉ đủ cả hai nguồn ở 17% số câu. Self-RAG viết lại truy vấn ở 11/35 câu và kết thúc bằng refuse ở 17 câu. Nếu chỉ tính các câu có

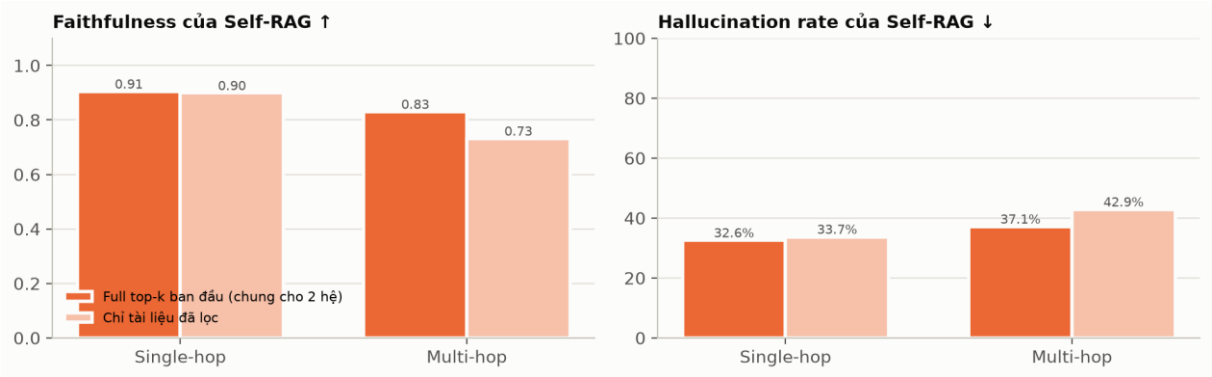
trả lời, hallucination rate của hai hệ là 83,3% và 72,2%: phần lớn mức giảm hallucination đến từ việc refuse, đổi lại độ đúng so với ground truth giảm từ 83% xuống 49%.

Bảng 10. Answer relevancy ở câu multi-hop: tính cả câu refusal (= 0) và chỉ tính câu có trả lời

Điều kiện	Hệ thống	Số câu có trả lời	Answer relevancy (cả câu refusal)	Answer relevancy (chỉ câu có trả lời)
top-k = 4	LangChain RAG	30/35	0,703	0,821
	LangGraph Self-RAG	18/35	0,419	0,815
top-k = 8	LangChain RAG	32/35	0,747	0,817
	LangGraph Self-RAG	18/35	0,420	0,816

Khi chỉ tính các câu có trả lời, answer relevancy của hai hệ chênh nhau rất ít (mô tả, không kiểm định vì tập câu trả lời của hai hệ khác nhau): không thấy dấu hiệu câu trả lời của Self-RAG kém liên quan hơn. Answer relevancy trung bình của Self-RAG thấp hơn chủ yếu vì câu refusal được 0 điểm theo định nghĩa của chỉ số, tức là đánh đổi giữa việc ít bịa và việc trả lời được nhiều câu.

Hallucination của RAG thuần ở câu multi-hop cao nhưng độ đúng vẫn cao (71% và 83%): khi top-4 thiếu một bài, mô hình bổ sung phần còn thiếu bằng kiến thức có sẵn. Câu trả lời có thể đúng nhưng không có căn cứ trong tài liệu truy xuất, nên vẫn bị tính là hallucination theo định nghĩa. Đây đúng là loại lỗi mà vòng tự kiểm tra của Self-RAG được thiết kế để chặn.



Hình 9. Faithfulness và hallucination rate của Self-RAG khi chấm trên full top-k và trên tài liệu đã lọc, top-k = 4

Ảnh hưởng của tài liệu dùng để chấm: nếu chấm Self-RAG trên tài liệu đã lọc, faithfulness ở câu multi-hop giảm từ 0,831 xuống 0,732 và hallucination rate tăng từ 37,1% lên 42,9%, vì nhiều claim được hỗ trợ bởi tài liệu đã bị loại ở bước lọc. Chấm trên cùng full top-k cho hai hệ tránh được sai lệch này.

Chi phí: Self-RAG gọi LLM trung bình 7,6 lần ở câu single-hop và 9,4 lần ở câu multi-hop (RAG thuần: 1 lần); thời gian mỗi câu là 26,4 giây so với 7,9 giây ở câu single-hop.

8. THẢO LUẬN VÀ HẠN CHẾ

- **Self-RAG có ích ở đâu:** lợi thế của Self-RAG chỉ xuất hiện khi tài liệu truy xuất không đủ để trả lời (câu multi-hop): vòng tự kiểm tra chặn được câu trả lời không có căn cứ, đổi lại hệ refuse nhiều hơn và trả lời đúng ít hơn. Khi tăng top-k lên 8, RAG thuần có đủ tài liệu hơn và lợi thế này thu hẹp, không còn statistical significance. Với câu hỏi dễ, nơi truy xuất đã tốt, không đủ bằng chứng để phân biệt chất lượng của hai hệ, còn Self-RAG tốn thời gian gấp 3,3 lần.
- **Giới hạn của bước lọc tài liệu:** grade_documents chấm từng đoạn riêng lẻ nên dễ loại đoạn chỉ chứa một phần thông tin của câu hỏi tổng hợp; chấm theo nhóm đoạn hoặc giữ lại đoạn của nhiều nguồn có thể cải thiện điểm này.
- **Giới hạn của cách cài đặt:** các bước tự đánh giá dùng mô hình 7B với prompt, không phải mô hình được huấn luyện riêng như Self-RAG gốc. Kết quả phản ánh cách cài đặt này, không phải giới hạn của phương pháp Self-RAG nói chung.
- **Judge:** llama3.1:8b có nhiễu khi chấm từng claim; chưa có đánh giá của con người để đối chứng. Kết luận chỉ dựa trên các chênh lệch có statistical significance.
- **Cỡ bộ câu hỏi:** 35 câu multi-hop và 23 câu out-of-corpus nên CI còn rộng; các chênh lệch nhỏ (dưới khoảng 15 điểm %) chưa phát hiện được.
- **Phần cứng:** GPU 6 GB chỉ chạy được mô hình 7–8B lượng tử hoá; mỗi lần đánh giá đầy đủ mất nhiều giờ.

9. KẾT LUẬN

Trên 144 câu hỏi, với top-k = 4: ở câu hỏi một đoạn, không đủ bằng chứng để phân biệt LangChain RAG và LangGraph Self-RAG trên cả ba chỉ số. Ở câu hỏi tổng hợp hai bài, Self-RAG giảm hallucination rate từ 71% xuống 37%, phần lớn nhờ refuse nhiều hơn (câu refusal có hallucination = 0; chỉ tính câu có trả lời, mức giảm nhỏ hơn nhiều, mục 7.4), và tăng faithfulness trên các câu cả hai hệ cùng trả lời, nhưng answer relevancy giảm từ 0,70 xuống 0,42 và độ đúng so với ground truth giảm, cũng do refuse nhiều hơn. Ở câu hỏi out-of-corpus, cả hai hệ đều refuse đúng toàn bộ. Self-RAG phù hợp khi ưu tiên tránh câu trả lời không có căn cứ hơn là trả lời được nhiều câu, với chi phí tính toán cao hơn.

Khi tăng top-k lên 8 (nhiều đoạn không liên quan), ở câu một đoạn Self-RAG có xu hướng ít hallucination hơn (28% so với 34%) nhưng chưa có statistical significance; ở câu tổng hợp, RAG thuần cải thiện nhờ có đủ tài liệu hơn nên chênh lệch hallucination rate thu hẹp (57% so với 40%, không có statistical significance), trong khi answer relevancy của Self-RAG vẫn thấp hơn rõ rệt.